



DEFINIÇÃO DAS CLASSES DE DOMÍNIO DA APLICAÇÃO

As classes de domínio representam as entidades centrais do ecossistema **Clyvo Paws**, modeladas utilizando o paradigma da Programação Orientada a Objetos (POO) e mapeadas para persistência relacional via Jakarta Persistence (JPA). Abaixo estão detalhadas as definições, propósitos, regras de integridade (*constraints*) e os relacionamentos de cada classe.

1. Tutor (**Tutor**)

- **Definição:** Representa o cliente da clínica veterinária, ou seja, o responsável legal pelos animais de estimação. É a entidade que interage com o ecossistema para gerenciar o cuidado dos pets.
- **Atributos Principais:** **id** (Identificador único), **nomeCompleto**, **cpf**, **telefone**, **email**, **senha**, **fotoUrl** e **endereço**.
- **Constraints (Restrições):**
 - **cpf:** Chave candidata única (*Unique Key*), com validação de formato de 11 dígitos numéricos. Não pode ser nulo (**@NotBlank**).
 - **email:** Deve ser único no sistema para fins de autenticação, validado com formato de correio eletrônico padrão (**@Email**).
- **Relacionamentos:** **1:N (Um para Muitos)** com a classe **Pet**. Um tutor pode ter vários pets cadastrados, mas um pet pertence obrigatoriamente a apenas um tutor.

2. Pet (**Pet**)

- **Definição:** É o núcleo central da jornada de saúde contínua do sistema. Representa o paciente animal atendido pela infraestrutura clínica.
- **Atributos Principais:** **id**, **nome**, **especie** (Enum), **raca**, **peso**, **sexo** (Enum), **dataNascimento**, **descricao** e **fotoUrl**.
- **Constraints (Restrições):**
 - **especie** e **sexo:** Uso estrito de Tipos Enumerados (Enum) para garantir a consistência taxonômica e biológica dos dados, impedindo inserções de textos inválidos.
 - **peso:** Deve ser um valor estritamente positivo (**@Positive**).
- **Relacionamentos:**
 - **N:1 (Muitos para Um)** com **Tutor** (Obrigatório, chave estrangeira **tutor_id**).
 - **1:N (Um para Muitos)** com **Consulta** (Um pet possui um histórico longitudinal de consultas).

3. Consulta (**Consulta**)

- **Definição:** Registra o evento clínico episódico e transacional realizado por um médico veterinário em um pet. É o ponto gerador do histórico longitudinal de saúde.
- **Atributos Principais:** **id**, **dataConsulta**, **clinica**, **nomeVeterinario** e **laudo**.
- **Constraints (Restrições):**
 - **dataConsulta:** Deve registrar o momento exato em que o atendimento ocorreu, utilizando **LocalDateTime**.
 - **laudo:** Campo de texto longo contendo o parecer médico e diagnóstico do profissional (**@NotBlank**).
- **Relacionamentos:**
 - **N:1 (Muitos para Um)** com **Pet** (Vínculo obrigatório com o paciente).
 - **1:N (Um para Muitos)** com **Medicamento** e **Agendamento**.

4. Medicamento (**Medicamento**)

- **Definição:** Representa o tratamento farmacológico ou preventivo prescrito ao animal durante uma consulta. Garante o acompanhamento da adesão medicamentosa.
- **Atributos Principais:** **id**, **nome**, **dosagem**, **frequencia**, **dataInicio**, **duracaoDias** e **status** (Enum).
- **Constraints (Restrições):**
 - **duracaoDias:** Deve ser um número inteiro maior que zero (**@Min(1)** ou **@Positive**).
 - **status:** Controlado via Enum (ex: ATIVO, CONCLUÍDO, INTERROMPIDO).
- **Relacionamentos:**
 - **N:1 (Muitos para Um)** com **Consulta** (Uma prescrição obrigatoriamente descende de um atendimento médico).
 - **1:N (Um para Muitos)** com **HistoricoDose**.

5. Histórico de Dose (**HistoricoDose**)

- **Definição:** Registra o "check-in" ou a confirmação de que o tutor administrou uma dose específica do medicamento prescrito no horário correto. Resolve a dor macro de esquecimento e abandono de tratamentos.
- **Atributos Principais:** **id** e **dataHoraToma**.
- **Constraints (Restrições):**
 - **dataHoraToma:** Registro temporal inviolável gerado no momento em que o usuário confirma a ação.
- **Relacionamentos:** **N:1 (Muitos para Um)** com **Medicamento**. Várias doses tomadas pertencem ao histórico de um único tratamento medicamentoso.

6. Agendamento (**Agendamento**)

- **Definição:** Representa os compromissos futuros preventivos ou de retorno clínico gerados automaticamente ou manualmente pelo sistema para combater a fragmentação do cuidado.
- **Atributos Principais:** **id**, **dataHora**, **titulo** e **descricao**.
- **Constraints (Restrições):**
 - **dataHora:** Deve ser obrigatoriamente uma data no futuro (**@Future**) em relação ao momento do agendamento.
- **Relacionamentos: N:1 (Muitos para Um)** com **Consulta**. O agendamento de retorno é originado a partir de uma consulta prévia que determinou a necessidade de reconsulta ou exames futuros.

7. Catálogo Preventivo (**CatalogoPreventivo**)

- **Definição:** Funciona como uma tabela dicionarizada de inteligência médica descritiva. Ela mapeia as predisposições de saúde e as diretrizes de medicina preventiva recomendadas para os animais.
- **Atributos Principais:** **id**, **especie** (Enum), **raca**, **doencaPredisposta**, **idadeAlertaMeses**, **dicaPrevencao** e **cuidadosRecomendados**.
- **Constraints (Restrições):**
 - Esta classe atua de forma isolada e desacoplada das tabelas transacionais de usuários. Seus registros são lidos pela aplicação para disparar alertas inteligentes no aplicativo com base na espécie informada pelo tutor no cadastro do pet.



EXPLICAÇÃO DOS RELACIONAMENTOS E INTEGRIDADE

- **Integridade Referencial (Foreign Keys):** No banco de dados físico, os relacionamentos **N:1 (@ManyToMany)** são garantidos por restrições de Chave Estrangeira (*Foreign Keys*) com a regra **ON DELETE RESTRICT** (padrão do JPA/Hibernate), impedindo, por exemplo, que um Tutor seja excluído se existirem animais vinculados ao seu cadastro, blindando a integridade do banco de dados.

- **Tipos Embutidos (Value Objects):** A classe **Tutor** faz uso do conceito de objeto embutido (**@Embedded**) para o **Endereco**. Isso significa que, conceitualmente na POO, são classes separadas para respeitar a alta coesão , mas no modelo de persistência física, os atributos de endereço tornam-se colunas integradas na tabela do tutor, otimizando o tempo de resposta da API por não exigir operações de junção (**JOIN**) no banco de dados.